

7.7 Case Study: Maintaining Access Frequencies

The positional list ADT is useful in a number of settings. For example, a program that simulates a game of cards could model each person's hand as a positional list (Exercise P-7.60). Since most people keep cards of the same suit together, inserting and removing cards from a person's hand could be implemented using the methods of the positional list ADT, with the positions being determined by a natural order of the suits. Likewise, a simple text editor embeds the notion of positional insertion and deletion, since such editors typically perform all updates relative to a *cursor*, which represents the current position in the list of characters of text being edited.

In this section, we will consider maintaining a collection of elements while keeping track of the number of times each element is accessed. Keeping such access counts allows us to know which elements are among the most popular. Examples of such scenarios include a Web browser that keeps track of a user's most accessed pages, or a music collection that maintains a list of the most frequently played songs for a user. We will model this with a new *favorites list ADT* that supports the size and isEmpty methods as well as the following:

`access(e)`: Accesses the element *e*, adding it to the favorites list if it is not already present, and increments its access count.

`remove(e)`: Removes element *e* from the favorites list, if present.

`getFavorites(k)`: Returns an iterable collection of the *k* most accessed elements.

7.7.1 Using a Sorted List

Our first approach for managing a list of favorites is to store elements in a linked list, keeping them in nonincreasing order of access counts. We access or remove an element by searching the list from the most frequently accessed to the least frequently accessed. Reporting the *k* most accessed elements is easy, as they are the first *k* entries of the list.

To maintain the invariant that elements are stored in nonincreasing order of access counts, we must consider how a single access operation may affect the order. The accessed element's count increases by one, and so it may become larger than one or more of its preceding neighbors in the list, thereby violating the invariant.

Fortunately, we can reestablish the sorted invariant using a technique similar to a single pass of the insertion-sort algorithm, introduced in the previous section. We can perform a backward traversal of the list, starting at the position of the element whose access count has increased, until we locate a valid position after which the element can be relocated.

Using the Composition Pattern

We wish to implement a favorites list by making use of a `PositionalList` for storage. If elements of the positional list were simply elements of the favorites list, we would be challenged to maintain access counts and to keep the proper count with the associated element as the contents of the list are reordered. We use a general object-oriented design pattern, the *composition pattern*, in which we define a single object that is composed of two or more other objects. (See, for example, Section 2.5.2.)

Specifically, we define a nonpublic nested class, `Item`, that stores the element and its access count as a single instance. We then maintain our favorites list as a `PositionalList` of *item* instances, so that the access count for a user's element is embedded alongside it in our representation. (An `Item` is never exposed to a user of a `FavoritesList`.)

```

1  /** Maintains a list of elements ordered according to access frequency. */
2  public class FavoritesList<E> {
3      // ----- nested Item class -----
4      protected static class Item<E> {
5          private E value;
6          private int count = 0;
7          /** Constructs new item with initial count of zero. */
8          public Item(E val) { value = val; }
9          public int getCount() { return count; }
10         public E getValue() { return value; }
11         public void increment() { count++; }
12     } //----- end of nested Item class -----
13
14     PositionalList<Item<E>> list = new LinkedPositionalList<>(); // list of Items
15     public FavoritesList() { } // constructs initially empty favorites list
16
17     // nonpublic utilities
18     /** Provides shorthand notation to retrieve user's element stored at Position p. */
19     protected E value(Position<Item<E>> p) { return p.getElement().getValue(); }
20
21     /** Provides shorthand notation to retrieve count of item stored at Position p. */
22     protected int count(Position<Item<E>> p) { return p.getElement().getCount(); }
23
24     /** Returns Position having element equal to e (or null if not found). */
25     protected Position<Item<E>> findPosition(E e) {
26         Position<Item<E>> walk = list.first();
27         while (walk != null && !e.equals(value(walk)))
28             walk = list.after(walk);
29         return walk;
30     }

```

Code Fragment 7.16: Class `FavoritesList`. (Continues in Code Fragment 7.17.)

```

31  /** Moves item at Position p earlier in the list based on access count. */
32  protected void moveUp(Position<Item<E>> p) {
33      int cnt = count(p);                                // revised count of accessed item
34      Position<Item<E>> walk = p;
35      while (walk != list.first() && count(list.before(walk)) < cnt)
36          walk = list.before(walk);                      // found smaller count ahead of item
37      if (walk != p)
38          list.addBefore(walk, list.remove(p));          // remove/reinsert item
39  }
40
41  // public methods
42  /** Returns the number of items in the favorites list. */
43  public int size() { return list.size(); }
44
45  /** Returns true if the favorites list is empty. */
46  public boolean isEmpty() { return list.isEmpty(); }
47
48  /** Accesses element e (possibly new), increasing its access count. */
49  public void access(E e) {
50      Position<Item<E>> p = findPosition(e);              // try to locate existing element
51      if (p == null)
52          p = list.addLast(new Item<E>(e));              // if new, place at end
53      p.getElement().increment();                        // always increment count
54      moveUp(p);                                         // consider moving forward
55  }
56
57  /** Removes element equal to e from the list of favorites (if found). */
58  public void remove(E e) {
59      Position<Item<E>> p = findPosition(e);              // try to locate existing element
60      if (p != null)
61          list.remove(p);
62  }
63
64  /** Returns an iterable collection of the k most frequently accessed elements. */
65  public Iterable<E> getFavorites(int k) throws IllegalArgumentException {
66      if (k < 0 || k > size())
67          throw new IllegalArgumentException("Invalid k");
68      PositionalList<E> result = new LinkedPositionalList<>();
69      Iterator<Item<E>> iter = list.iterator();
70      for (int j=0; j < k; j++)
71          result.addLast(iter.next().getValue());
72      return result;
73  }
74  }

```

Code Fragment 7.17: Class FavoritesList. (Continued from Code Fragment 7.16.)

7.7.2 Using a List with the Move-to-Front Heuristic

The previous implementation of a favorites list performs the $\text{access}(e)$ method in time proportional to the index of e in the favorites list. That is, if e is the k^{th} most popular element in the favorites list, then accessing it takes $O(k)$ time. In many real-life access sequences (e.g., Web pages visited by a user), once an element is accessed it is more likely to be accessed again in the near future. Such scenarios are said to possess *locality of reference*.

A *heuristic*, or rule of thumb, that attempts to take advantage of the locality of reference that is present in an access sequence is the *move-to-front heuristic*. To apply this heuristic, each time we access an element we move it all the way to the front of the list. Our hope, of course, is that this element will be accessed again in the near future. Consider, for example, a scenario in which we have n elements and the following series of n^2 accesses:

- element 1 is accessed n times.
- element 2 is accessed n times.
- ...
- element n is accessed n times.

If we store the elements sorted by their access counts, inserting each element the first time it is accessed, then

- each access to element 1 runs in $O(1)$ time.
- each access to element 2 runs in $O(2)$ time.
- ...
- each access to element n runs in $O(n)$ time.

Thus, the total time for performing the series of accesses is proportional to

$$n + 2n + 3n + \cdots + n \cdot n = n(1 + 2 + 3 + \cdots + n) = n \cdot \frac{n(n+1)}{2},$$

which is $O(n^3)$.

On the other hand, if we use the move-to-front heuristic, inserting each element the first time it is accessed, then

- each subsequent access to element 1 takes $O(1)$ time.
- each subsequent access to element 2 takes $O(1)$ time.
- ...
- each subsequent access to element n runs in $O(1)$ time.

So the running time for performing all the accesses in this case is $O(n^2)$. Thus, the move-to-front implementation has faster access times for this scenario. Still, the move-to-front approach is just a heuristic, for there are access sequences where using the move-to-front approach is slower than simply keeping the favorites list ordered by access counts.

The Trade-Offs with the Move-to-Front Heuristic

If we no longer maintain the elements of the favorites list ordered by their access counts, when we are asked to find the k most accessed elements, we need to search for them. We will implement the `getFavorites( $k$ )` method as follows:

1. We copy all entries of our favorites list into another list, named `temp`.
2. We scan the `temp` list k times. In each scan, we find the entry with the largest access count, remove this entry from `temp`, and add it to the results.

This implementation of method `getFavorites( $k$ )` takes $O(kn)$ time. Thus, when k is a constant, method `getFavorites( $k$ )` runs in $O(n)$ time. This occurs, for example, when we want to get the “top ten” list. However, if k is proportional to n , then the method `getFavorites( $k$ )` runs in $O(n^2)$ time. This occurs, for example, when we want a “top 25%” list.

In Chapter 9 we will introduce a data structure that will allow us to implement `getFavorites` in $O(n + k \log n)$ time (see Exercise P-9.51), and more advanced techniques could be used to perform `getFavorites` in $O(n + k \log k)$ time.

We could easily achieve $O(n \log n)$ time if we use a standard sorting algorithm to reorder the temporary list before reporting the top k (see Chapter 12); this approach would be preferred to the original in the case that k is $\Omega(\log n)$. (Recall the big-Omega notation introduced in Section 4.3.1 to give an asymptotic lower bound on the running time of an algorithm.) There is a specialized sorting algorithm (see Section 12.3.2) that can take advantage of the fact that access counts are integers in order to achieve $O(n)$ time for `getFavorites`, for any value of k .

Implementing the Move-to-Front Heuristic in Java

We give an implementation of a favorites list using the move-to-front heuristic in Code Fragment 7.18. The new `FavoritesListMTF` class inherits most of its functionality from the original `FavoritesList` as a base class.

By our original design, the access method of the original class relies on a protected utility named `moveUp` to enact the potential shifting of an element forward in the list, after its access count had been incremented. Therefore, we implement the move-to-front heuristic by simply overriding the `moveUp` method so that each accessed element is moved directly to the front of the list (if not already there). This action is easily implemented by means of the positional list ADT.

The more complex portion of our `FavoritesListMTF` class is the new definition for the `getFavorites` method. We rely on the first of the approaches outlined above, inserting copies of the items into a temporary list and then repeatedly finding, reporting, and removing an element that has the largest access count of those remaining.

```

1  /** Maintains a list of elements ordered with move-to-front heuristic. */
2  public class FavoritesListMTF<E> extends FavoritesList<E> {
3
4      /** Moves accessed item at Position p to the front of the list. */
5      protected void moveUp(Position<Item<E>> p) {
6          if (p != list.first())
7              list.addFirst(list.remove(p));           // remove/reinsert item
8      }
9
10     /** Returns an iterable collection of the k most frequently accessed elements. */
11     public Iterable<E> getFavorites(int k) throws IllegalArgumentException {
12         if (k < 0 || k > size())
13             throw new IllegalArgumentException("Invalid k");
14
15         // we begin by making a copy of the original list
16         PositionalList<Item<E>> temp = new LinkedPositionalList<>();
17         for (Item<E> item : list)
18             temp.addLast(item);
19
20         // we repeated find, report, and remove element with largest count
21         PositionalList<E> result = new LinkedPositionalList<>();
22         for (int j=0; j < k; j++) {
23             Position<Item<E>> highPos = temp.first();
24             Position<Item<E>> walk = temp.after(highPos);
25             while (walk != null) {
26                 if (count(walk) > count(highPos))
27                     highPos = walk;
28                 walk = temp.after(walk);
29             }
30             // we have now found element with highest count
31             result.addLast(value(highPos));
32             temp.remove(highPos);
33         }
34         return result;
35     }
36 }

```

Code Fragment 7.18: Class FavoritesListMTF implementing the move-to-front heuristic. This class extends FavoritesList (Code Fragments 7.16 and 7.17) and overrides methods moveUp and getFavorites.